



MOHANBIR SAWHNEY

Microsoft Corporation: Branding and Positioning .NET

As he drove on the I-520 bridge toward Redmond on an early Monday morning in February 2002, John Williams, senior director of marketing for Microsoft's .NET ("dot net") initiative, considered his company's branding, positioning, and messaging strategy for .NET.

Microsoft's .NET was a comprehensive family of next-generation software products and tools that would allow information, devices, and people to connect in a unified, personalized manner. This connectivity was enabled by the emerging XML (extensible markup language) standard, which could be used to create XML Web services—small, discrete, building-block applications that could communicate with each other as well as with other, larger applications—via the Internet. Microsoft was betting that .NET would become the foundation for all its future products, and would propel Microsoft's future growth in the enterprise software market.

Microsoft had officially announced .NET in the summer of 2000. For the first eighteen months the company targeted its marketing efforts at software developers. In pursuing this strategy, Microsoft hoped to build momentum in developing new applications that would take advantage of .NET in the enterprise market. In January 2002 Microsoft broadened the message by making .NET the primary focus of its \$200 million "Agility" mass media campaign targeted at business decision makers (BDMs) and technology decision makers (TDMs). This campaign used the concept "One Degree of Separation" to communicate the value proposition of system interoperability as an enabler of business agility. In targeting the BDMs and TDMs, Microsoft's goal was to create "pull" for .NET-enabled products and solutions.

Williams had recently rejoined Microsoft and was tasked with clarifying the .NET offering branding and communication strategy. He realized that this task involved three key sets of decisions. First, he had to define exactly what .NET was, and communicate this definition effectively within the company, as well as to external audiences. Was .NET a programming model, a platform, or a technology? As .NET represented a blend of complex technology concept changes, rapidly evolving technologies, and a host of potential benefit areas, there was a lot of confusion about what .NET really was. Further complicating the issue, some business units within Microsoft had begun attaching the .NET name to their products—for varying reasons, and with varying customer implications. Should this be allowed to continue?

Second, Williams had to determine the appropriate branding strategy for .NET. Should .NET be branded as an umbrella brand that spanned multiple products? Should it instead be considered

an ingredient brand, which powered or enabled other products? Or should .NET merely be a sub-brand, attaching itself to existing Microsoft brands?

Finally, Williams faced the challenge of finding a positioning and messaging strategy that could span the diverse segments in his audience. Developers had different needs and were motivated by different benefits from .NET than the TDMs and BDMs. Could a single positioning strategy be used to communicate with each audience? If so, what value propositions could be designed for each segment that would still be consistent with the overall positioning strategy, and how could these value propositions be communicated without conflicting with Microsoft's existing brands or alienating any specific constituency?

Microsoft Overview

Microsoft was the worldwide leader in software, services, and Internet technologies for personal and business computing. The company offered a wide range of products and services designed to empower people through software—any time, any place, and on any device. Microsoft was founded as a partnership on April 4, 1975, by William H. Gates III and Paul G. Allen, and incorporated on June 25, 1981. Headquartered in Redmond, Washington, the company operated subsidiary offices in more than sixty foreign countries and employed nearly 48,000 people worldwide. In the fiscal year ended June 30, 2001, Microsoft reported net revenues of \$25.3 billion and net income of \$7.35 billion. Microsoft was one of the world's largest corporations, with a market capitalization of almost \$335 billion in early February 2002. **Exhibit 1** provides historical financial results for Microsoft.

Microsoft developed, manufactured, licensed, and supported a wide range of software products for a multitude of computing devices. Microsoft software included operating systems for servers, personal computers, and intelligent devices; server applications for client/server environments; knowledge worker productivity applications; and software development tools. The company's online businesses included the MSN network of Internet products and services. Microsoft also licensed consumer software programs, sold hardware devices, and provided consulting services.

In the twenty-seven years since its inception, Microsoft had grown from a small programming languages developer into the largest software company in the world. The company achieved dominance in the desktop and enterprise operating systems and platforms, desktop productivity applications, development tools, and online applications and services. **Exhibit 2** shows a chronological list of important events in Microsoft's evolution.

Microsoft's business strategy emphasized the development of a broad line of software products for information technology (IT) professionals, knowledge workers, developers, and consumers, marketed through multiple channels of distribution. The company was structured around the following core groups: the Business Groups; the Worldwide Sales, Marketing, and Services Group; Microsoft Research; and the Operations Group. **Exhibit 3** describes Microsoft's core business groups.

The Development of .NET

In June 2000 Microsoft announced an ambitious initiative for the next generation of software that would revolutionize the way its software was designed, developed, and used. This initiative, called Microsoft .NET, was designed to take advantage of the emerging Internet standards for XML Web services, allowing the creation and use of software that would connect information, devices, and people in a unified, personalized way.

.NET was a catch-all name for a comprehensive family of products, built on industry and Internet standards, that provided for each aspect of developing (tools), managing (servers), using (building block services and smart clients), and experiencing (rich user experiences) XML Web services. Microsoft planned to embed .NET technology into all of its applications, tools, and servers, as well as in new products that would take advantage of XML Web service capabilities (see **Exhibit 4** for a visual depiction of .NET tools and components).

From the very outset, Microsoft's .NET strategy had been rather ambiguous. Confusion about its content puzzled analysts, reporters, rivals, and even Microsoft employees. So, what exactly was .NET? The options included, but were not limited to:

- A platform¹
- A framework²
- An operating system³
- Development tools
- Software
- Services
- Products and applications
- XML Web services
- Some or all of the above

Microsoft's own announcements had taken varying positions on the nature of .NET over the eighteen months since its announcement, leading up to the present advertising campaign.

Summer 2000—First Announcement

On June 22, 2000, signaling a new era of personal empowerment and business opportunity for consumers, businesses, and software developers, Microsoft unveiled the vision and road map for its next generation of software and services, the Microsoft .NET platform. Capitalizing on the explosion of Internet-based computing and

¹ Software architecture or operating system that can be used to run and compile programs and applications and include development tools.

² Architecture, programming model, building blocks for application development.

³ The master control program that runs the computer. The first program loaded when the computer is turned on, its main part, the "kernel," resides in memory at all times. The operating system sets the standards for all application programs that run in the computer. The applications "talk to" the operating system for all user interface and file management operations.

communications, Microsoft .NET (pronounced “dot-net”) will provide easier, more personalized, and more productive Internet experiences by harnessing constellations of smart devices and Web sites with advanced software through Internet protocols and formats.⁴

Spring 2002—Revised Description

The original message from Microsoft in 2000 was altered and modified several times during the next two years. Microsoft’s 2002 .NET message stated the following:

Microsoft .NET is software that connects information, people, systems, and devices.

.NET connects a broad range of personal and business technologies, enabling you to access and use important information, whenever and wherever it is needed. Built on XML Web services standards, .NET enables both new and existing applications to connect with software and services across platforms, applications, and programming languages.

.NET is integrated across the Microsoft platform, providing the ability to quickly build, deploy, manage, and use connected, secure solutions with XML Web services. These solutions enable faster, more agile business integration, and the promise of information anytime, anywhere, on any device.⁵

Microsoft .NET as a set of software technologies was based on two pillars: XML Web services and the .NET programming model.

XML WEB SERVICES

XML was revolutionizing the way applications talked to other applications—or more broadly, how computers talked to other computers—by providing a universal data format that allowed data to be easily adapted, transformed, and used by disparate devices. XML could be invoked across platforms and operating systems, regardless of programming language. XML could be used to create XML Web services.

A Web service was any piece of software that was made available over the Internet and used a standardized XML messaging system. Unlike traditional software applications, Web services were designed to be loosely coupled reusable software components that let applications share data. More importantly, Web services could invoke capabilities from other applications without regard to how those applications were built, what operating system or platform they ran on, and what devices were used to access them. While XML Web services remained independent of each other, they could loosely link themselves into a collaborating group to perform a particular task. **Exhibit 5** gives an example of how Web services could be used to improve the process of inventory management.

Web services would allow organizations to communicate data without intimate knowledge of each other’s IT systems, and without human intervention. Further, because all communication was in XML, Web services were not tied to any one operating system or programming language.

⁴ Key Events in Microsoft History, <http://www.microsoft.com/mscorp/museum/KeyEventsInMicrosoftHistory.doc>.

⁵ <http://www.microsoft.com>.

Web services used four open standards to communicate with other Web services or applications:

- eXtensible Markup Language (XML) to tag the data
- Simple Object Access Protocol (SOAP) to transfer the data
- Web services Definition Language (WSDL) to describe the services available
- Universal Directory Definition Interface (UDDI) to list what services are available

These standards were defined through public standards organizations such as the World Wide Web Consortium (W3C).

.NET PROGRAMMING MODEL

The .NET programming model was used for modeling, building, deploying, and running XML Web services and applications. It provided software developers with a standards-based, multi-language environment for integrating existing applications and platforms with next-generation applications and services built as Web services.

.NET-Related Products

The Microsoft .NET offering included a comprehensive family of products—clients, services, servers, and tools—designed to support XML Web services.

DEVELOPMENT TOOLS

Microsoft Visual Studio .NET and the Microsoft .NET Framework provided solutions for developers to build, deploy, and run XML Web services.

XML WEB SERVICES

Microsoft was also developing a core set of XML Web services that could be combined with other XML Web services or software to create a single solution. One example of these solutions was Microsoft MapPoint .NET, an XML Web service that enabled integration of maps, driving directions, and other location intelligence into applications, business processes, and Web sites.

SERVERS

Microsoft provided server infrastructure—including the Microsoft Windows 2000 server family and the .NET enterprise servers—for deploying, managing, and orchestrating XML Web services.

SMART CLIENTS

Turning these different pieces of software/services into a seamless computing experience for the end user was made possible by using Microsoft .NET to build rich applications that integrated with Windows and the Internet. Microsoft .NET provided this experience in the form of smart client software, through PCs, mobile and embedded devices, Microsoft Windows XP, Windows XP Embedded, and Windows CE .NET.

The Strategic Importance of .NET

.NET was perhaps the most important strategic initiative for Microsoft since it launched the Microsoft Windows operating system. At stake was the future of the enterprise software market. Microsoft saw .NET as an opportunity to defend its position in the platforms world against potential threats from XML Web services, and to penetrate the enterprise computing business where Microsoft had a limited presence relative to IBM and Sun Microsystems.

XML Web Services as the Next Generation of Computing

XML Web services represented what might become the next paradigm shift in the computing world. The computing world had evolved from the first generation of mainframe computing, through desktop computing, client-server, and finally browser computing. While there was widespread agreement that XML Web services would play an important role in the next (fifth) generation of computing, there was considerable debate over exactly what this next generation would look like. One vision, championed mainly by IBM, saw the future in terms of a concept called “grid computing.” The competing vision, promoted by Microsoft, envisaged XML Web services as enablers of enterprise and global class computing with agility and connectivity in development, deployment, and integration of applications and services.

IBM'S VISION: GRID COMPUTING AND “E-BUSINESS ON DEMAND”

According to IBM, the paradigm shift in computing would be grid computing and Web services.

IBM proclaimed grid computing⁶ as the successor to Web-based computing. Grid computing was a layer of software and services that would sit on top of operating systems and link different systems together, allowing them to share resources. By adding the ability to extensively share computing power, applications, and storage to the Web's current ability to share text and multimedia files, problems requiring a lot of computing resources could be solved, devices could work past their own limits, and collaboration could become more intense. XML Web services standards would play a critical role in grid computing, enabling disparate layers of software and services to communicate and integrate seamlessly.

For IBM, the notion of grid computing and Web services formed the basis of its latest service offering: “e-business on demand” as “the next utility.” IBM promoted a service by which it would provide computing, applications, infrastructure, and IT services (100 percent outsourcing of enterprise IT needs) over the Internet as a service on demand. These services would be priced on a “pay-as-you-go” basis, a model similar to paying for utilities like electricity and water.

The grid computing model would be a threat to Microsoft because its business anchor had always been its operating systems dominance. Microsoft's lock on Windows had enabled it to extract tremendous economic profits from the desktop and enterprise business. Microsoft Windows alone accounted for almost \$8 billion in revenues, nearly a third of Microsoft's total revenues of \$25.3 billion in fiscal year 2001. Further, Microsoft's Office family of products,

⁶ IBM Global Services, Executive Tek Report (November 13, 2001), Grid computing: Distributed advantage, http://www-1.ibm.com/services/files/etr_grid.pdf.

whose revenues were tightly linked to the Windows platform, accounted for another \$9.5 billion in revenues (see Exhibit 3 for details).

Within the grid computing vision, XML Web services would threaten Microsoft's position in server operating systems. XML Web services allowed applications to share data and invoke capabilities from other applications without regard to how those applications were built, what operating system or platform they ran on, and what devices were used to access them. The same Web service could be implemented, used, and provided on a Microsoft, UNIX, Mainframe, or Linux platform, using .NET or J2EE⁷ or any other development technologies. If Web services were decoupled from the operating system and platform used to develop and run applications, the operating system could become less relevant in the enterprise computing world.

MICROSOFT'S VISION: XML WEB SERVICES AND "BUSINESS AGILITY" FOR THE ENTERPRISE

Microsoft's vision relied on the belief that enterprise applications, data, and knowledge were core assets of the enterprise. In an environment where intellectual property was becoming the only source of sustainable competitive advantage, Microsoft did not believe that enterprises would outsource all their IT platforms and applications to service providers like IBM. Rather, Microsoft believed that enterprises would continue to build and implement internal enterprise applications. The major change would be in the ability to easily connect, integrate, share, and change current and future applications in enterprise and global class computing.⁸

In this scenario, enterprise computing would be centralized at the enterprise level, rather than outsourced and consumed as a "shared utility." Therefore, enterprises would require more sophisticated development, environments, servers, and tools that would ensure scalability, reliability, agility, and connectivity. Microsoft's challenge would be to ensure the integration of XML Web services and .NET services with its core assets: the Windows operating system platform.

.NET as Microsoft's Entry Point into Enterprise and Global Computing⁹

Enterprise and global class application developments had become a high-profile battleground between two standards: J2EE and .NET. Just as J2EE was backed, developed, and implemented by powerhouses such as IBM, Sun Microsystems, BEA, Hewlett-Packard, and Oracle, .NET was developed and run solely by Microsoft. While J2EE applications and services could be developed, implemented, and run on almost any platform (UNIX, Windows, and Linux), using multiple vendors (IBM, BEA, Hewlett-Packard, Sun, and Oracle, to name a few), .NET was tightly coupled with Microsoft platforms. **Exhibit 6** summarizes the strengths and weaknesses of the J2EE platform relative to the .NET platform.

⁷ Java 2 Platform, Enterprise Edition. A platform from Sun Microsystems for building Web-based enterprise applications.

⁸ Global class computing is a style of computing that focuses on strategies that go beyond just the enterprise level into multienterprise, global strategies. Global computing supports the idea that a solution must not only be enterprise-capable, it must also be capable of dealing with: (1) multienterprise strategies, (2) the culture of the consumer, and (3) globally sensitive issues (that is, multilingual and multicultural issues). For example, issues of enterprise class computing frequently deal with reliability, serviceability, and availability. This typically occurs in situations where one enterprise controls the resources and the access to them. However, with the advent of the Internet, that scenario is increasingly less realistic. As enterprises integrate their systems with other enterprises' systems, and as interoperability across the Internet increases in importance, the guarantee of flexible integration and relationship management will become even more paramount than the guarantee of reliability, serviceability, and availability.

⁹ "Microsoft and Global Class Computing—Changing the Game," *Inside Gartner* XVIII, no. 15 (April 10, 2002): 9–11.

J2EE was perceived as a leading platform for enterprise applications that were built *to task* (specifically to provide a specific function or process), and *to last* by providing scalability and reliability over time. However, J2EE was not built for future changes or the integration of those enterprise applications—key challenges that organizations faced.

According to John Williams, Microsoft .NET had three advantages over J2EE in the competition to achieve enterprise and global-class solutions:

1. It delivered technologies that touched consumers and could increase productivity in a collaborative sense.
2. It delivered systems that could be easily reconfigured for a variety of opportunistic needs.
3. It provided a seamless integration interface with current and future applications, allowing these systems to be built for change.

As a result of these capabilities, .NET represented an opportunity for Microsoft to penetrate and gain momentum in the lucrative enterprise computing business. The research firm Gartner predicted that Microsoft would make substantial progress in adding enterprise-class capabilities to its products, and that Microsoft could lead the delivery of global-class solutions by 2006 (see **Exhibit 7**).

For defensive as well as offensive reasons, .NET was critical to the continued success and growth of Microsoft. If Microsoft could promote and lead the industry toward adoption of its vision of XML Web services, it would be able to maintain its dominant position in operating systems, platforms, desktops, and enterprise servers—the historical and current source of its success. Further, Microsoft could use .NET to enter and gain momentum in the lucrative enterprise and global computing markets that IBM had historically dominated.

Evolution of .NET Branding and Messaging

Historically, Microsoft's advertising campaigns were designed for autonomous business units and product groups. Advertising budgets were assigned to each product group to promote their efforts, and little was done to coordinate an overriding corporate theme.

All this changed with the launch of the "Agility" campaign in 2001, which was Microsoft's first unified message about the relevance of its software to large enterprises. National commercials, print ads, and billboards were used to deliver the message that "Microsoft builds software that makes companies more agile in a competitive workplace."

The Agility campaign was designed as an umbrella message to support the evolution of the .NET strategy. It was backed with a \$200 million budget.

Later in 2001 the campaign evolved from a purely branding message to a message focusing on the capabilities of the .NET enterprise server and platform. In hindsight, Microsoft executives believed that television may have been the wrong vehicle to depict these products' association to the overarching Agility strategy.

The platform approach was confusing to developers. On one hand .NET was all about Web services and interoperability/integration, while on the other hand .NET was about a new programming model for building standalone Windows applications. Which one was it?

In an effort to clarify the message, in January of 2002 Microsoft launched the third phase of the Agility campaign, “One Degree of Separation.” The campaign message claimed, “When software lets you quickly give customers what they want, that’s one degree of separation. That’s business with .NET from Microsoft.” (See **Exhibit 8** for the press release announcing the third phase of the Agility campaign, focused on the enterprise market.)

This was the first time in the Agility campaign that .NET was referenced in conjunction with Web services, and its ability to flexibly connect businesses with their customers and partners. .NET was no longer referred to as a platform, but instead as software and services.

While Microsoft had traditionally targeted developers and IT professionals, this third phase attempted to target business decision makers as well. Unfortunately, business decision makers did not understand what Web services were and how they could make them more powerful in dealing with customers and competitors. Without understanding what Web services were, BDMS had a difficult time understanding the benefits they provided, and why they needed .NET at all. Furthermore, a television broadcast campaign was not an adequate vehicle for offering detail and messaging as to how .NET interacted with Windows and Microsoft’s other software products like Office.

Williams knew something had to be done. He had to clearly define .NET and its benefits to several unique audiences. Most importantly, he needed to create a positioning plan that would clarify .NET’s role in Microsoft’s overall strategy while simultaneously strengthening the Windows value proposition.

Alternative Branding Approaches

Technology firms had historically underutilized the concept of branding. The misconception that sophisticated users were not susceptible to the influence of brands was the mistaken rationale behind this trend. However, technology firms like IBM, Intel, and Hewlett-Packard were beginning to realize the power of branding as a way to rise above the noise and confusion in technology markets, and to provide continuity across generations of rapidly evolving technology products.

Coordinating a brand strategy in a large technology company like Microsoft with a diverse product portfolio was a complex task. Microsoft needed to create a branding strategy that would span overlapping products without confusing customers or diluting the positioning of individual products. Williams remembered from his marketing classes at business school that technology firms could choose from three different approaches to branding their product portfolios. These approaches differed on the extent to which the focus of the branding effort was the company versus the product. They also differed on the role that the brand played in the customer purchase decision. The options Williams was examining for branding .NET were umbrella branding, ingredient branding, and sub-branding.

.NET as an Umbrella Brand

Williams considered the advantages of creating an umbrella branding campaign for .NET. The .NET strategy was Microsoft's vision of the future, and all new and existing products would be greatly impacted and enhanced by .NET features. A .NET umbrella brand would state that all Microsoft products are .NET first, with other features second. Every product would be branded as ".NET" (e.g., .NET Platforms, .NET Servers, .NET Applications, .NET Services, and .NET Tools). This strategy would emphasize the strategic role for .NET across the entire company, and would permit a great degree of flexibility and consistency across all the advertising campaigns for specific products.

However, Williams was unsure if a .NET umbrella would replace or compete with the existing Windows umbrella brand, or for that matter, the Microsoft umbrella brand. He was unsure if introducing a new umbrella would impact the current Microsoft brand equity. Any negative impact on the Microsoft corporate brand and the Windows umbrella brand would have to keep in mind that these brands were extremely successful and valuable. According to a 2001 study by the leading brand valuation firm Interbrand, the Microsoft corporate brand was the most valuable technology brand in the world, and the second most valuable brand in the world, with an estimated value of \$65 billion in 2001. The umbrella branding strategy would also be very expensive and time-consuming, because it would involve creating and positioning an entirely new umbrella brand from scratch.

.NET as an Ingredient Brand

Williams also considered positioning .NET as an ingredient brand. This approach would allow Microsoft to position .NET as a part of every current and new Microsoft product without confusing or erasing existing brand equity. Advertising campaigns could use the .NET ingredient approach to highlight different benefits for different audiences. Consumers should be familiar with this type of ingredient, which, like "Intel Inside," makes existing programs and products work better. In contrast with past ingredient branding campaigns, .NET would assume an ingredient role in not only other original equipment manufacturers' smart clients or third parties' services and offerings, but also in future Microsoft offerings. The ingredient branding approach would be consistent with the view that, at its heart, .NET simply was an enabling technology that was used in Microsoft's products, not a product by itself. It would also mitigate the risk and expense of launching an umbrella brand.

One of the problems with this approach, however, was that .NET was not just an ingredient—it was a new company-wide strategy, and positioning it as an ingredient might downplay the significance of .NET in Microsoft's future. Additionally, an ingredient approach might not highlight exactly how new and different .NET was. The final problem with this approach was that it could be confusing—the target audiences might not understand how one ingredient could work for all of Microsoft's products, from Windows to Office to SQL Server to Xbox.

.NET as a Sub-Brand

The final approach Williams considered was positioning .NET as a sub-brand. This strategy would allow the existing umbrella brands to be the "driver" brands, while .NET would take on the secondary role to suggest that .NET was a new technology that made existing Microsoft products

better. For instance, Windows could be branded as Windows .NET, and Microsoft Office could be called Office .NET. In fact, Microsoft had already used this approach for the release of its .NET-enabled programming tools, called Visual Studio .NET.

The sub-branding strategy would easily identify the “newness” of .NET, and would create a new and common vocabulary around the product line. It would be a logical evolution of the chronological numbering that Microsoft and other technology companies followed (Windows 98, Windows XP, Windows .NET). It would also allow the current umbrella brands to remain the driver brands.

One problem with this approach was that .NET was not a single product or technology. Rather, it was an overarching set of tools and features affecting all existing Microsoft products. Additionally, this approach would create confusion within Microsoft, where employees understood that .NET impacted all of their products. Finally, positioning .NET as a sub-brand would dilute the power of the .NET message externally. If .NET was its own brand, then how did it affect Windows or Office? If it affected all products, then why was it a separate brand?

Positioning and Value Proposition Design for .NET

As Williams considered the branding strategy, he also needed to consider the diverse audiences for the .NET message. His analysis suggested that there were five major customer audiences who Microsoft needed to communicate with in its .NET campaign:

- Programmers (developers)
- Business decision makers
- Technical decision makers
- Employees (within corporations)
- End consumers

Williams believed that it was often difficult to distinguish among these audiences in any given media or marketing activity, and that the best option would be to find an overarching positioning framework that could transcend all of these audiences—one that could permit tailored value propositions to be constructed for each audience, but still be consistent with the overall positioning framework.

Williams had commissioned qualitative research to understand the extent to which the different audiences understood the concepts of XML and XML Web services, and the benefits of .NET that would most likely appeal to them. The research indicated a significant split between the concepts and benefits that the technical audiences (developers, IT professionals, and TDMs) identified with and those that the BDMs identified with. The key findings from the qualitative research are summarized in **Exhibit 9**.

Williams had also commissioned quantitative research to understand the extent to which various audiences understood what .NET was and what they would use .NET for. This research also tested how these different audiences responded to key descriptors of .NET, and various benefit statements for .NET. Consistent with Williams’s expectations, various audiences seemed

to respond to the benefit statements quite similarly, illustrating the challenge of customizing the value propositions and messaging to each audience. However, Williams wondered if this finding was partly attributable to the fact that the three benefit statements tested were not as different as they could have been. **Exhibit 10** summarizes the findings from the quantitative research.

Williams believed that the value proposition for each of the audiences should focus on how .NET enables connections for them. Developers needed to understand how .NET would make it easier to write software applications that would work with and connect with any other applications. IT managers needed to understand how .NET enabled systems to interoperate and connect with each other. BDMs needed to understand how .NET could help people work together both within and outside their companies with partners, suppliers, and customers. Employees, and eventually consumers, needed to understand how connecting their different devices and data would make their lives easier.

Williams believed that this new focus on business benefits was a strategic change for Microsoft, and he wanted to see this approach succeed. Historically Microsoft had used advertising campaigns mainly to tout its technology prowess, not to describe how using the technology would benefit its customers. Williams believed that the right positioning approach could guarantee success for the third phase of the Agility campaign, and would ultimately encourage Microsoft to continue its advertising spotlight on real benefits.

As he considered the importance of succeeding with .NET branding, he wondered whether Microsoft should try to reach both application developers (programmers) and decision makers with the same message. This approach would guarantee a consistent .NET message, and would certainly be more affordable. However, the five .NET target audiences would probably not make their decisions based on the same information. Additionally, he knew that customers think in terms of activities, not products or features, and he was unsure how one message could speak to the activities of both programmers and decision makers.

Williams also wondered if Microsoft should attempt to reach all its target audiences simultaneously, or if it should have audience-specific campaigns with more targeted messaging. The benefit of segmented campaigns would be the possibility of timing the advertising campaigns so that different audiences received customized messages at the appropriate points in time. This could reduce the chances that, for instance, consumers would see products that they could not yet use. As of February 2002 .NET consumer products and services were not yet widely available in the market, so premature advertising and “hype” could create unrealistic customer expectations. Microsoft was sensitive to criticism that it often pre-announced products that were little more than “vaporware.”

However, Williams understood from past experiences the power of mind-share marketing. Building the right brand equity in advance of a product or service launch could serve as a preemptive strategy to avoid market share competition once the products did reach the marketplace. Further, an integrated and coordinated campaign would reduce the confusion that had already been created by uncoordinated and inconsistent messaging. Williams was concerned that a segmented approach to messaging would create further confusion about what .NET was, and how Microsoft wanted customers to think about the .NET value proposition.

Williams contemplated which media vehicle would be the best to reach .NET’s target audiences. To date, Microsoft had used national television spots, advertisements in print magazines, and a presence at trade events. The national TV component accounted for 20 to 25 percent of the Agility campaign reach objective. Print campaigns reached 60 percent of the target;

event, online, and direct mail comprised the remaining 15 to 20 percent. Given his new branding and positioning strategies, Williams wondered which marketing methods would reach his target audience with the greatest return on his advertising investment.

Conclusion

As he reached his office, Williams looked over some recent press articles that illustrated the urgency of the branding and positioning problem for .NET. An excerpt from a *Wall Street Journal* article noted:

*Tim Maleeny, a vice president at Publicis & Hal Riney, a unit of France ad agency Publicis Groupe, believes Microsoft still has to work to get away from ad campaigns centered on individual product launches, instead of broader branding. Indeed, Microsoft officials say the current campaign is in many ways pegged to its launch this week of a software-development product called Visual Studio.NET. . . . Maleeny praise[s] Microsoft rival IBM as a company that has managed to run coherent and easy-to-understand campaigns that put complicated issues, such as new online-business services, in a wider context.*¹⁰

As a result, Microsoft was faced with the challenge of changing the following general impression about .NET, according to an industry expert:

Current ruminations about .NET in various forums are reminiscent of the fable of the three blind men attempting to identify an elephant: It's perceived as very different things, depending on your perspective. Some see .NET as Microsoft's next-generation Visual Studio development environment. Some see it as yet another new programming language. Some see it as a new data-exchange and messaging framework, based on XML and SOAP. In reality, .NET wants to be all of these things, and a bit more.

Williams had to make a presentation to Steve Ballmer, the CEO of Microsoft, in two weeks. Ballmer had indicated that he wanted a clear set of recommendations from Williams. There was no time to lose.

¹⁰ Rebecca Buckman, "Geeky Microsoft to Unveil Simple Ads," *Wall Street Journal*, February 15, 2002.

Exhibit 1: Microsoft Historical Financial Results (\$ in billions)

Microsoft's revenue and income for the past ten fiscal years is summarized below:

Financial Year Ending	Net Revenue	Percent Growth	Net Income	Percent Growth
6/30/1992	2.78	50	0.708	53
6/30/1993	3.79	36	0.953	35
6/30/1994	4.71	25	1.150	20
6/30/1995	6.08	29	1.450	27
6/30/1996	9.05	49	2.200	51
6/30/1997	11.94	32	3.450	57
6/30/1998	15.26	28	4.490	30
6/30/1999	19.75	29	7.790	73
6/30/2000	22.96	16	9.420	21
6/30/2001	25.30	10	7.350	(22)

Source: Microsoft Annual Report

Exhibit 2: Chronology of Key Events in Microsoft's History

- 1975** The company created its version of BASIC (its first product) for a computer created by MITS (Micro Instrumentation and Telemetry Systems) of Albuquerque, New Mexico, and was later used for other computer manufacturers.
- 1978** Microsoft announced its third language product, Microsoft COBOL-80, the year its sales exceeded \$1 million for the first time.
- 1981** Microsoft launched MS-DOS on IBM's personal computer, the first step into its Windows dominance.
- 1983** Microsoft launched its Microsoft Mouse and Word program.
- 1985** Microsoft debuted its Windows software system.
- 1986** Microsoft went public.
- 1987** Windows 286 and Windows 386 were announced, as well as Windows 2.0 and Microsoft Excel (spreadsheet) for Windows.
- 1988** SQL Server (the database server) was announced.
- 1989** Microsoft Word for Windows began shipping and the Microsoft Office productivity suite made its debut as a strategic move from OS and development tools to the productivity applications market.
- 1990** Microsoft became the first software company to reach revenues of more than \$1 billion. During that year Microsoft Windows 3.0 was announced and sold more than 4 million copies.
- 1991** Microsoft announced Visual BASIC and a new version of Excel 3.0.
- 1992** Windows 3.1 was announced.
- 1993** Windows users exceed 25 million, making it the most popular graphical operating system. Designed for mission-critical corporate applications, Microsoft Windows NT was launched.
- 1994** Microsoft Office suite became the leader in its category, with sales of more than 7 million units.
- 1995** Microsoft Windows 95 made its debut, and for the first time a Microsoft product was named with a year tag (a policy that was later used with Windows 98 and 2000). MSN, the Microsoft Network online service, was also launched in the same year.
- 1996** More than 30 million people used Microsoft Excel (making it the most popular spreadsheet); MSNBC, the 24-hour news and information network from NBC News and Microsoft, debuted.
- 1997** Internet Explorer 4.0 was launched.
- 1998** For the first time, Exchange Server outsold Lotus Notes, and Windows 98 was launched.
- 1999** Microsoft launched Internet Explorer 5.0 and MS Office 2000.
- 2000** Microsoft launched its first separate Windows product lines for home users (ME) and business/professional users (2000), targeting different Windows audiences with different product lines.
- 2001** Following the apparently unsuccessful strategy, Microsoft launched the XP generation of Microsoft products (targeting both home and office users), led by Office XP and Windows XP, as the part of Microsoft's strategy with which individual users directly interact.

Exhibit 3: Microsoft's Business and Product Groups**BUSINESS GROUPS AND PRODUCT SEGMENTS**

The company's product segments, which are based on the business groups, are Desktop and Enterprise Software and Services; Consumer Software, Services, and Devices; Consumer Commerce Investments; and Other.

(a) The Desktop and Enterprise Software and Services Segment

The Desktop and Enterprise Software and Services segment includes the Platforms Group and the Productivity and Business Services Group. The Platforms Group is responsible for continuing to evolve the Windows platform. In addition, the division includes the .NET Enterprise Server Group, the Developer Tools Division, and the Windows Digital Media Division. The Productivity and Business Services Group drives Microsoft's broad vision for productivity and business process applications and services. This group includes the Office Division, the Emerging Technologies Group, the Business Tools Division, and the Business Applications Group, which includes Central and Microsoft Great Plains. Desktop and Enterprise Software and Services revenue was \$22.41 billion in 2001, including Desktop Applications (\$9.54 billion), Desktop Platforms (\$8.04 billion), and Enterprise Software and Services (\$4.83 billion).

(b) The Consumer Software, Services, and Devices Segment

The Consumer Software, Services, and Devices segment contains the MSN Business Group, the Personal Services Business Group, and the Home and Retail Division. MSN Business Group runs the network programming, business development, and worldwide sales and marketing for MSN and Microsoft's other services efforts, including MSN eShop, the MSNBC venture Slate, and MSNTV. The Personal Services Group (PSG) focuses on making it easier for consumers and businesses to connect online and to deliver software as a service on a variety of devices. PSG encompasses Microsoft's Personal .NET initiative, the Services Platform Division, the Mobility Group, the MSN Internet Access, Consumer Devices Group, and the User Interface Platform Division. The Home and Retail Division develops and markets learning and entertainment software and the Xbox game console. Consumer Software, Services, and Devices revenue was \$1.95 billion in 2001.

(c) The Consumer Commerce Investment Segment

The Consumer Commerce Investment segment includes Expedia, Inc.,¹¹ the HomeAdvisor™ online real estate service, and the MSN CarPoint® online automotive service. Consumer Commerce Investments revenue was \$299 million in 2001.

(d) The Other Segment

The Other segment includes Microsoft Press and Hardware. Other revenue was \$630 million in 2001.

¹¹ Microsoft had recently completed the sale of its holding in Expedia.

Exhibit 3 (continued)

MICROSOFT'S WORLDWIDE SALES, MARKETING, AND SERVICES GROUP

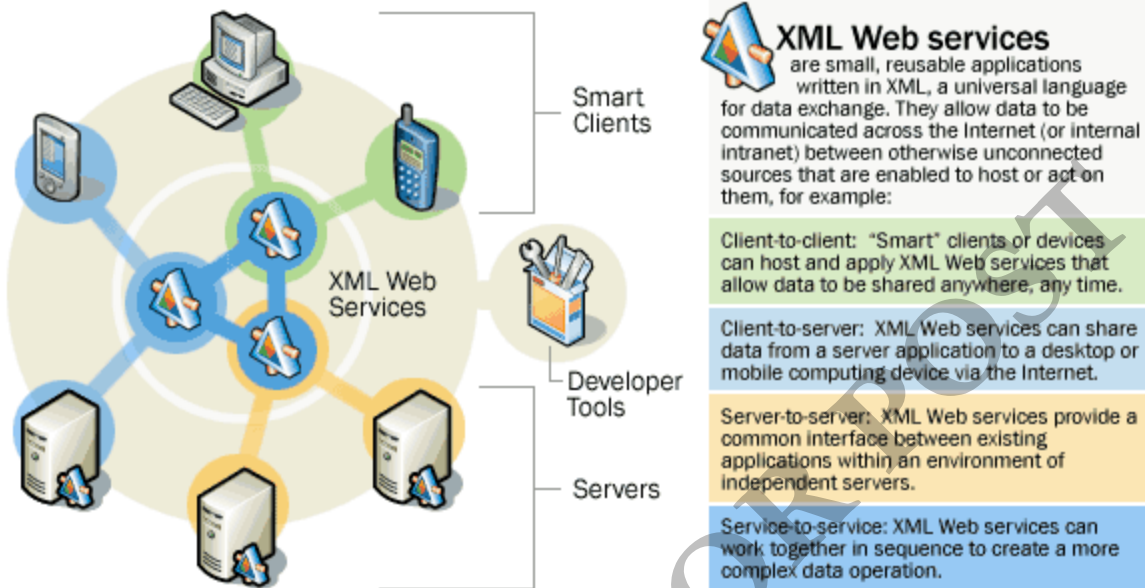
The Worldwide Sales, Marketing, and Services Group integrates the activities of Microsoft's sales and service partners with the needs of Microsoft customers around the world. In addition, the group includes Microsoft Product Support Services, the Network Solutions Group, the Enterprise Partner Group, the Central Marketing Organization, and all three of Microsoft's major business-sales regions worldwide.

MICROSOFT RESEARCH

Microsoft Research works on devising innovative solutions to computer science problems, such as making computers easier to use, designing software for the next generation of hardware, improving the software design process, and investigating the mathematical underpinnings of computer science.

THE OPERATIONS GROUP

The Operations Group is responsible for managing business operations and overall business planning. This includes corporate functions such as finance, administration, human resources, and information technology.

Exhibit 4: Key Components of .NET

Source: Microsoft

Exhibit 5: XML Web Service for Inventory Management

Say you have an inventory system. If you do not connect it to anything else, it is not very valuable—you can track inventory, but it is a lot of work, and what that one system can do all by itself is limited. For instance, every item you sell needs to be entered not only into your inventory system, but also separately into your accounting system and your customer account records. You also need to remember to order more of that item from your suppliers next time you place an order. The cost/benefit ratio is largely unappealing, because the gains from the system are close to the overhead costs of using it.

However, if you connect your inventory system to your accounting system with XML, it gets more interesting. Now, whenever you buy or sell something, the implications for your inventory and your cash flow can be tracked in one step. If you go further, and connect your warehouse management system, customer ordering system, supplier ordering systems, and your shipping company with XML, suddenly that inventory management system is worth a lot. You can do end-to-end management of your business while dealing with each transaction only once, instead of once for every system it affects. A lot less work and a lot less opportunity for errors.

These connections can be made easily using XML Web services. XML Web services allow the applications to share information via the Internet, regardless of the operating system or back-end software that the application is using.

Source: Microsoft

Exhibit 6: Comparison of Strengths and Weaknesses of J2EE versus .NET

	J2EE	.NET
Weaknesses	- Standards lag product features, reducing portability and leverage - Weaker platform integration - Poor multilanguage support - Risk of need for GUI applications - High-end J2EE servers cost more than .NET	- Single vendor lock-in - Major platform change introduces stability and conversion risks - Technical lag drives up development cost - Weaker production application support - Platform elements more shallow - Standards religion not proven
Strengths	- Platform choice for reliability, scale-up, and legacy - Greater vendor leverage - Stronger architecture focus, interfaces, and clustering - Tools more team-oriented - Greater adoption rate/openness - Greater platform depth	- Best GUI development - Tighter platform integration and unified management - Better, cheaper tools - Multilanguage and cross-language support - More developers (but many are not "enterprise class") - ASP.NET better than JSPs (for now)

Source: Mike Gilpin, "The New Application Platform: J2EE and Web Services Standards Evolution Driving Application Agility and Integration," Giga Information Group, April 2002.

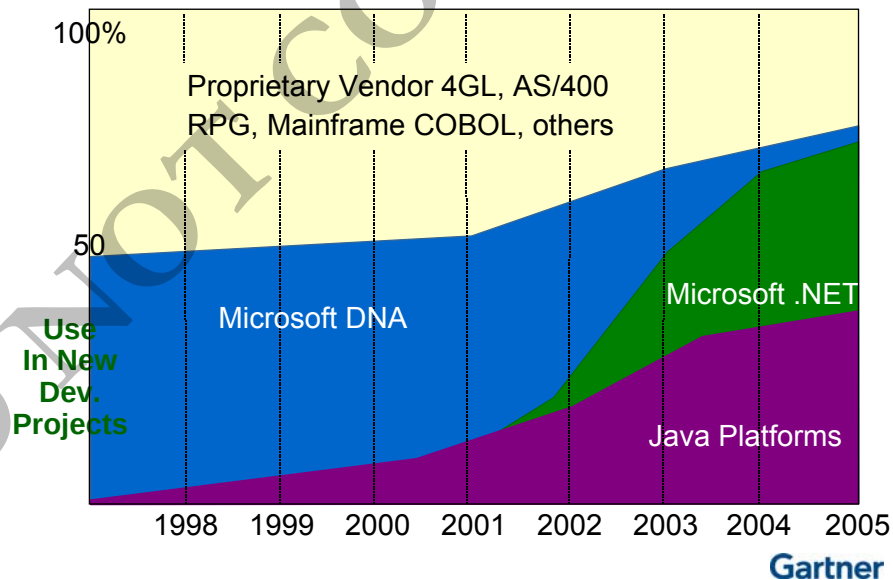
Exhibit 7: Prediction of Share of .NET versus Java (J2EE) Programming Models

Exhibit 8: Microsoft Press Release Announcing .NET Advertising Campaign

MICROSOFT ANNOUNCES NEW BUSINESS POSITIONING FOCUSED ON ENTERPRISE CUSTOMERS

Launches \$200 Million U.S. Advertising Campaign Around "Software for the Agile Business"

REDMOND, Wash.—Jan. 21, 2001—Microsoft Corp. today launched a major business positioning initiative that underscores the unique benefits that Microsoft® software delivers today and will deliver to an even greater degree with the Microsoft .NET platform—to meet the rapidly changing requirements of large enterprise customers.

Built around the theme of Software for the Agile Business, the company's new marketing effort spotlights how Microsoft software provides businesses with the agility to turn the accelerating pace of change into a competitive advantage.

As part of the new strategy, Microsoft is launching a \$200 million U.S. advertising campaign that showcases how Microsoft software provides businesses with enterprise-level quality and reliability, unprecedented speed and flexibility, and the empowerment solutions they require to compete and succeed in today's digital economy.

The launch of Microsoft's Agility campaign focuses attention on the increasing competition between Microsoft and Sun Microsystems Inc., Oracle Corp., and IBM Corp. in the enterprise software segment, which Microsoft estimates is approximately \$90 billion annually.

"Businesses today need to be fast and nimble to compete and respond to customer needs, yet a lot of the technology for business is big, slow, and expensive," said Steve Ballmer, president and CEO of Microsoft. "We believe Microsoft offers a better and far more cost-effective solution. Over the past ten years, we have listened to customers and focused like a laser beam on building enterprise software that gives businesses the agility they need to compete in today's marketplace. The ad campaign we are launching today is designed to show enterprise customers why they should take a good look at our solutions."

With the launch of the Microsoft Windows® 2000 operating system and the line of Microsoft .NET enterprise servers, Microsoft's enterprise products not only match competitors' in breadth and depth, but go far beyond by providing solutions that enable customers to adapt quickly and flexibly to fast-changing business conditions and customer needs. Agility is also a core benefit of Microsoft's .NET strategy, which is focused on delivering the technologies for the next-generation Internet.

"Microsoft has an amazing growth opportunity in the enterprise," said Mich Mathews, vice president of marketing at Microsoft. "The challenge now is to make sure all our customers know that we have solutions that are superior and offer greater value. That's the principal goal of this marketing effort."

The Home Shopping Network (HSN) is one of many customers Microsoft is working closely with to develop and deploy cutting-edge technology solutions. With more than 750,000 hits per day on its Web site, HSN needed an IT infrastructure that would enable it to respond quickly to marketplace demands and solve performance and availability problems easily. By migrating from an Oracle and Sun solution and standardizing on a Microsoft and Intel solution, HSN reduced costs, increased performance, simplified vendor management, and decreased time-to-market for new functionality.

"Using software solutions from Microsoft has significantly enhanced our ability to build improvements into our operations and remain ahead of our competition," said Scott Mitchell, chief technology officer at Home Shopping Network Interactive. "The combination of Microsoft's integrated products and services also enabled us to capitalize on software we already had, which resulted in significant cost savings. Microsoft technology also helped us move quickly. We rebuilt our entire Web site from the ground up in less than 45 days."

Microsoft solutions deliver the agility to successfully meet and adapt to evolving business and customer needs that a wide range of global companies require, including Guinness World Records, Haystack Toys, JobNet, World Chinese Commercial Bank, Telenor, Koninklijke Vendex, Telecom Malaysia, Lycos, and Mortgage Ramp.com (a division of GMAC).

Founded in 1975, Microsoft (Nasdaq "MSFT") is the worldwide leader in software, services, and Internet technologies for personal and business computing. The company offers a wide range of products and services designed to empower people through great software—any time, any place, and on any device.

Exhibit 9: Excerpts from Qualitative Research on XML Web Services and .NET

OVERALL REACTIONS

- *For technical audiences (developers, IT, and partners), understanding what XML Web services are and how they differ from XML was key to understanding the .NET concept. Providing detail on what the industry standards are and the role they play was critical for these audiences to understand the difference between XML and XML Web services. It was also important to clarify how the data exchange happens, and that it will not affect one's data infrastructure. Until they had enough information to understand how it worked, and therefore believe that it could work, they were unable to get excited about the concept or relate it to their own work environment.*
 - o In general, those who had more experience with programming languages, especially Web-based database applications, were consistently the individuals who best grasped the concept of XML Web services and ultimately .NET. Therefore, Web developers tended to understand the concept the best and IT implementers the least. Experience with XML helped also, but since most had a limited view of XML, it tended to also limit their view of XML Web services (unless they really grasped what Web services were).
 - o The language used in the descriptions sometimes limited the credibility of the concepts or the audience's ability to understand them. Phrases that over-promised or sounded too "blue sky" increased skepticism and weakened the credibility of the idea. Phrases that sounded marketing oriented, like Microsoft jargon, or provided only vague benefits did not add value to the communication and, in some cases, detracted from the audience's understanding of the idea.
 - o Technical audiences wanted information and scenarios that were grounded in realism and that clearly spoke to them as technical professionals. This meant not only sticking to technical language, but also adjusting to their level of strategic business concerns. For example, TDMs and partners have more organizational-level concerns than do IT implementers and developers.
 - o The following were important in helping technical audiences understand XML Web services and .NET:
 - Emphasizing and explaining the role of the XML Web standards (the descriptions shown in the groups were fairly effective in doing this).
 - Clarifying that XML is a communications layer that works on top of existing systems and does not affect the underlying data structure format or infrastructure. Therefore, all aspects of the system do not need to be exposed to get the data interchange.
 - Emphasizing that other companies would provide development platforms for XML Web services, and that .NET products would work with those platforms. This helped clarify that .NET is one of many platforms that will utilize open technology standards to enable the development and use of XML Web services.

Exhibit 9 (continued)

- Examples/scenarios that help people envision the new possibilities and benefits that XML Web services and .NET provide. For example, scenarios indicating how the whole external/internal model can “take down the walls between companies” would be particularly compelling. The more they address a relevant problem or issue and clearly illustrate the things that are not possible today, the better. It is also important for technical audiences to know that it was not necessary for Microsoft to “hold their hand” in order to implement what was described in the scenarios, but that Microsoft did and will provide the knowledge and support needed.
- *Nontechnical audiences (BDMs and knowledge workers), on the other hand, had no desire for detailed technical information.* They do not relate well to the concept of back-end data exchange between servers or applications nor do they want or need to know how a technology works to find value in it. What they want/need are examples and scenarios that are easy to understand and believable, with relevant end user benefits that seem easy to realize (i.e., it will not take a lot of training, time, or money to get the benefit).
 - Scenarios that emphasized the idea of a single transaction automatically sending off information to update other computers and Web sites helped people to visualize the interconnectivity that .NET provides in a way that was relevant to them.
 - Being able to easily link your company’s systems with systems outside the company (in the Dollar Rent-A-Car scenario) was another scenario that helped people see the possibilities of .NET.
 - The degree to which a scenario described something that could not be done today, but that was clearly beneficial and seemed easy to be able to do, the greater the positive impact. Complex scenarios often made nontechnical people feel that the technology would be too complex to use.

CONCERNS/BARRIERS

- *Security* was an issue in all groups and related to a presumed increase in the vulnerability of one’s data because others can easily access it, as well as the impact of viruses on totally integrated systems. Due to the significance of this concern, security in an interconnected world should be proactively addressed. For example, the single sign-on benefit should not be touted without providing a clear explanation of why this does not pose a greater security risk than usual.
- *Lack of control* was of particular concern among nontechnical audiences who believed that it would be hard to protect access to proprietary data or that they would provide or receive information they did not want. It is helpful to directly address this by emphasizing how preferences and directives work to control who sees the data.

Exhibit 9 (continued)

- *Data integrity* concerns were driven by the perceived catastrophic effect of errors in a totally integrated system, as well as a perceived inability to keep data intact if anyone had the ability to access, manipulate, and edit data. Emphasizing the difference between allowing data to be manipulated and allowing it to be changed at the source helps address how data integrity will be maintained.
- *Too blue sky*. Most technical audiences felt that true materialization of XML Web services as a standard was years from becoming a reality due to the magnitude of the effort needed to set global standards across industries and get people to follow them.
- *Has been promised before*. Technical audiences (especially developers) were skeptical as to whether XML Web services would truly have longevity or whether it will just be another promise of revolutionary change, like Java, that will not deliver.
- *Seems difficult to use*. Nontechnical audiences were concerned that this would not be simple for end users to use without training, and this detracted from its appeal. Related to this is the fact that ITs and developers believed that it is critical that the end user get on board with this technology for it to have longevity. It is possible to address this issue by directly communicating that the technology is virtually transparent to the end user—that is, except for setting preferences by simply checking boxes when prompted, the computer/back end does all the work. Leveraging the fact that Microsoft products are easy to use could also provide necessary reassurance.
- *How it works*. Nontechnical audiences were often confused about how different companies work together in an XML Web services world. They wondered what would happen if everyone did not opt in and how that would affect their own business situation.
- *Job security and loss of the human element*. If systems were able to communicate so much better without human intervention, some perceived their jobs would be threatened and that they would lose control over technology, as it would be doing all the thinking for them. General knowledge workers primarily expressed both of these concerns.

MICROSOFT AS A PROVIDER OF .NET

- *Both technical and nontechnical audiences felt that if any company could pull off a concept like .NET, it would be Microsoft*. Positive reactions were based on Microsoft's position as an industry leader with significant influence and clout, a technological leader that stays on top of new developments, and having easy-to-use products that would enhance adoption of .NET at the end-user level.
- At the same time, Microsoft being behind .NET also raised serious concerns:
 - o *Trust*. Technical people and BDMs had doubts regarding Microsoft's real dedication to open standards and ability to "play well with others" due to its historically proprietary nature. This was a key concern given that without this, the basic premise of .NET would not work. Some even saw an inherent contradiction in .NET: if all products/platforms easily work together, how can Microsoft's products be "optimized" for .NET? Specific things that can help mitigate these concerns include:

Exhibit 9 (continued)

- Explicitly communicating how .NET works across platforms and with non-.NET platforms, products, and services. In respondents' words, "don't make .NET the only real choice." Part of this includes addressing the inherent contradiction that Microsoft will integrate completely with anyone, but will integrate "better" with other .NET products/services.
 - Make an honest effort to work with existing technologies and integrate what is already out there rather than replacing everything with new components (this was a frequent appeal from tech audiences).
 - Get partners on board and let everyone know who Microsoft is partnering with.
 - Continuously emphasize and give examples of the fact that Microsoft is dedicated to the open standards, while being careful not to add/develop proprietary extensions.
- o *Need to partner.* Because of the magnitude of the undertaking, as well as concerns about Microsoft's willingness to play fair, technical audiences strongly felt that Microsoft must partner with other key industry players. Part of Microsoft's challenge will be to partner on and promote XML Web services without coming off as self-serving or dishonest. A potentially viable strategy would be for Microsoft to be upfront about realizing they cannot do this alone. The reality seems to be that the more Microsoft can get other major industry players on board with XML Web services, the greater their own chances of success will be with .NET. Partnering with others would also lend greater credibility to the "fairness" of .NET.
 - o *Reliability.* All audiences voiced concerns of reliability with Microsoft products. A bug in a totally integrated system is perceived to be devastating, so Microsoft would need to "get it right the first time" and deviate from its typical mode of releasing several versions or service packs before the bugs are worked out.
 - o *Credibility.* Many asked for further clarification of Microsoft's vision for how they plan to go from where they are now to where they need to be. This included addressing the magnitude of the effort required to set global standards across industries and get people to follow them, and providing realistic timing as to when it is expected to move from theory to reality.

BENEFITS OF .NET

- *Knowledge workers found value in the benefits relating to total integration of their information and their devices, sometimes referring to .NET as "one-stop shopping."* They found personal control to be an important benefit, as it addressed their earlier concerns about privacy and security. Also compelling was the concept of mobility, which they interpreted as being able to access both their personal and work-related information from virtually any computer and any location. TSKWs (Technologically Savvy Knowledge Workers) particularly preferred benefits relating to increased productivity, adaptability, and information access.

Exhibit 9 (continued)

- *BDMs responded most enthusiastically to benefits that spoke to their most pressing issues and needs—making it easier to connect with their customers and saving money by making integration across internal and external systems faster and cheaper.* The integration of legacy systems, in particular, was a particularly motivating benefit (although somewhat unbelievable). Second-tier benefits were those that increase the power and productivity of their employees, such as empowering employees to make better decisions, increasing worker mobility, and improving teamwork and collaboration. Making technology a competitive advantage was too vague and did not deliver a tangible benefit.
- *Developers generally focused on the efficiencies .NET would provide for them: faster, easier solutions that translate to increased ROI.* This included the ability to reuse services and distribute them to any device, make changes easily, and leverage existing skills. Web developers saw value for their customers in terms of richer, more flexible solutions. Corporate developers were interested in integrating more easily across disparate systems and internal maintenance. The value of smart clients and tailored services was not readily apparent or as broadly motivating.
- *Most IT respondents focused on easing the role of IT with faster, easier integration and the ability to “build once, use many.”* TDMs also looked to more business-oriented benefits such as saving time and money, ability to choose the best vendor, and better use of data by users. Providing user mobility and using agile systems, which is indicative of flexibility for modifications and integration, were also key benefits to IT.
- *Partners saw both the benefits of faster development time by reusing services and reducing redundancy, as well as the cost savings implications.* They also saw compelling business benefits such as IT staff reductions and selling more products and services. Employee-oriented benefits were less relevant to this audience. While partners are potentially effective advocates for .NET due to their strong relationship with Microsoft and their senior-level positions, this research clearly indicated that they must be “sold” into the vision like any other developer or IT professional.

Source: Microsoft Internal Research

Exhibit 10: Excerpts from Quantitative Research on .NET Positioning and Messaging

Understanding .NET

- When people were then exposed to descriptions of .NET, there were likewise few who did not understand

Top Testing .NET Descriptors Ranked by Understand Very Well	Overall Total NOT Understand
.NET is Microsoft's platform for XML Web services, the next generation of software that connects our world of information, devices, and people in a unified, personalized way. .NET will become part of the Microsoft applications, tools, and servers you already use today – as well as new products that extend XML Web service capabilities to your personal and business needs.	8
.NET is Microsoft's platform for XML Web services. .NET products and services securely draw on other services through the Internet to accomplish tasks or deliver data to a range of devices and locations – home or work PCs, servers, cell phones, pagers, PDAs, or other future devices, providing access to a single set of customized functionality and data.	13
.NET is Microsoft's platform for XML Web services, the next generation of software that connects our world of information, devices, and people in a unified, personalized way. .NET encompasses a full range of software to ensure all kinds of technology products can securely communicate with and utilize XML Web Services – from handheld devices or TV set top boxes, to business server infrastructure, and every type of device and application in-between.*	13

What Would You Use .NET For?

Open End Response	Overall	IT	DEV	BDM	KW
Websites: Building/designing websites / Adding tools to the website / XML, etc.	7	7	11	4	6
Everything / All applications / A lot of things	6	4	8	6	6
For information/data: Integration / Management / Gathering	6	2	6	9	6
Connecting the company / Connecting with other locations / Communication/information sharing with other locations/colleagues	5	7	5	7	2
Connect/integrate applications/systems / Integrating various platforms, etc.	5	6	5	4	5
Integration of devices / Connecting to hand held devices / Integrating wireless devices with computer applications, etc.	5	5	2	7	6
Applications: Building/developing applications / Web-based applications	5	4	7	2	4
Business and personal	5	3	5	3	8
For business	5	3	5	2	6
Cross platform / Information sharing regardless of platform	3	4	2	3	1
Connecting to clients / Communication/information sharing with customers/vendors	3	4	2	2	2
Remote/mobile access / Telecommuting	3	1	2	2	5
E-business, E-commerce, Business to Business	1	3	1	1	1
Nothing/Would not use it	4	5	7	3	2
Others	10	11	9	10	9
DK/No response	14	15	9	21	15

Source: Microsoft Internal Research

Exhibit 10 (continued)

Top Scoring .NET Descriptors

Index=Very Interested + Understand Very Well +Very Believable	Index	Corp BDM	ITDM	Corp Dev	ISV/ VAR	TS KW	Gen KW
.NET is Microsoft's platform for XML Web services. .NET products and services securely draw on other services through the Internet to accomplish tasks or deliver data to a range of devices and locations – home or work PCs, servers, cell phones, pagers, PDAs, or other future devices, providing access to a single set of customized functionality and data.	128	139	163	132	143	116	74
.NET is Microsoft's platform for XML Web services, the next generation of software that connects our world of information, devices, and people in a unified, personalized way. .NET encompasses a full range of software to ensure all kinds of technology products can securely communicate with and utilize XML Web Services – from handheld devices or TV set top boxes, to business server infrastructure, and every type of device and application in-between.*	127	125	158	130	149	100	90
.NET is Microsoft's platform for XML Web services, the next generation of software that connects our world of information, devices, and people in a unified, personalized way. .NET will become part of the Microsoft applications, tools, and servers you already use today – as well as new products that extend XML Web service capabilities to your personal and business needs.	125	127	144	125	132	115	100

Top Scoring .NET Benefit Statements

Index=Very Interested + Understand Very Well +Very Believable	Index	Corp BDM	ITDM	Corp Dev	ISV/ VAR	Gen KW
Removes Barriers. Using .NET products, you can "wrap" your existing business applications and processes with XML Web services, which means they'll work with programs, services, and applications built on other platforms, so there won't be any technology barriers between you and your customers.	116	118	135	107	106	110
Agility/Familiar. (IT/BDM ONLY) Your employees already know Microsoft software. Because .NET will be built into all new Microsoft products, using .NET will increase the efficiencies of your IT staff and end users with technologies that make systems easier to integrate, deploy, manage, and use.	115	121	119	-	-	-
Privacy/Security. .NET gives you an incredible choice of Internet-based services that can perform tasks on your behalf while you maintain complete control of your data. You determine what information about you is stored, where it's stored, how it's used, who can access it, etc., and you can easily change these preferences anytime you want.	115	123	128	105	103	119

Source: Microsoft Internal Research

Case Preparation Questions

1. Evaluate the various options for defining .NET. Should .NET be defined as a platform, a framework, a set of products and tools, or something else? Or should it be some combination of these? How are internal Microsoft audiences likely to react to your definition? How about external audiences?
2. Evaluate the three branding options for .NET. Which approach would you favor—umbrella branding, ingredient branding, or sub-branding? Why? Define how existing Microsoft products and brands would relate with .NET in the approach you recommend. What hurdles will you face in implementing your chosen branding approach?
3. Should .NET be positioned using an umbrella positioning framework, or should it be positioned differently for different audiences (IT pros, developers, TDMs, BDMs, and knowledge workers)? If you recommend the umbrella approach, how would you make the messaging relevant to diverse audiences? If you recommend a segmented approach, how would you ensure consistency?
4. Use the research results in Exhibit 9 and Exhibit 10 to define the positioning strategy and value propositions for .NET. What do you learn from this research? What do you still not know?